

Visual Workflow Builder

& Hook Configuration System

Technical Architecture, Diagrams & Implementation Guide

Architecture

Hook Schema

GQL Resolvers

Workflow Builder

Pre/Post Hooks

Table of Contents

Document Overview

#	Section	Page
1	System Architecture Overview	3
2	Hook Lifecycle & Execution Flow	4
3	Hook Schema Structure	5
4	Visual Workflow Builder UI	6
5	Pre & Post Hook Reference	7
6	GQL Resolver Catalogue	8
7	Available Hook Functions	9
8	Workflow Requirements Matrix	11
9	Hook Schema Quick Reference	12
10	Implementation Checklist	13

1. System Architecture Overview

End-to-end request flow across all system components

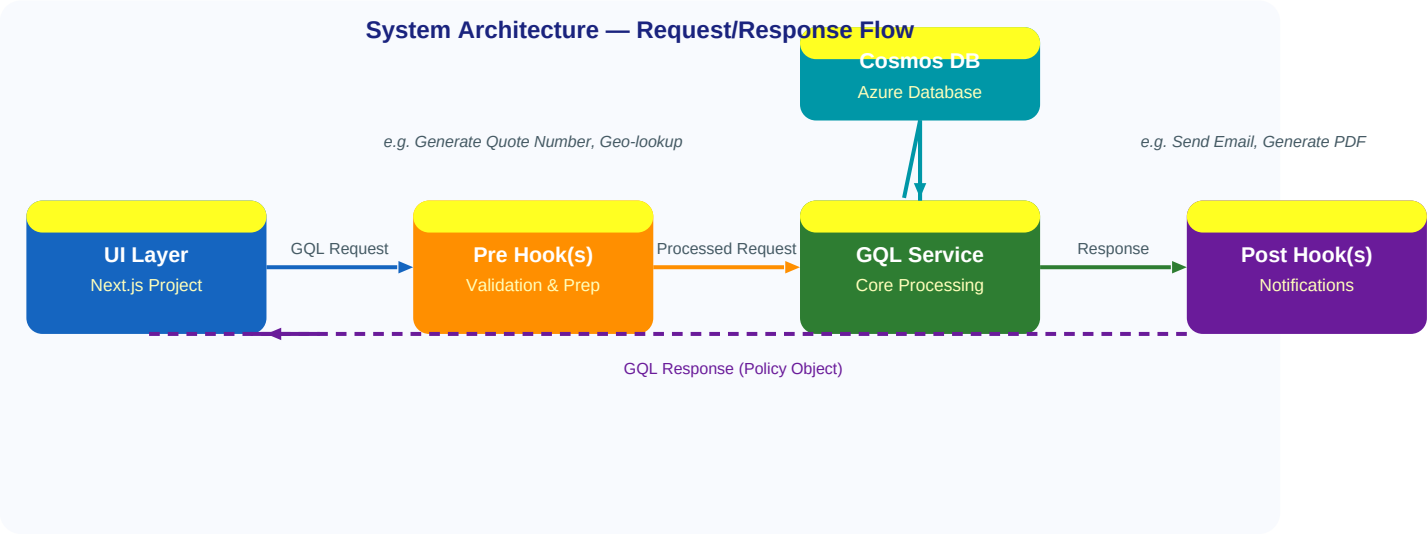


Figure 1: The system follows a linear request pipeline. The Next.js UI emits a GraphQL (GQL) request which passes through Pre Hooks for preparation tasks (e.g., generating quote numbers, geo-lookups). The processed request then hits the GQL Service which reads/writes to Cosmos DB. After processing, Post Hooks fire for secondary tasks such as email notifications and PDF generation. The final GQL Response (typically the Policy Object) is returned to the UI.

Component Responsibilities

Component	Technology	Role	Key Interactions
UI Layer	Next.js / React	Renders forms, triggers GQL requests	Sends/receives GQL to/from Hook layer
Pre Hooks	Node.js modules	Pre-process requests before DB writes	Receives raw GQL, outputs enriched payload
GQL Service	GraphQL / Apollo	Core resolver logic, DB orchestration	Reads/writes Cosmos DB, triggers Post Hooks
Post Hooks	Node.js modules	Side-effects after main processing	Sends emails, generates documents, updates status
Cosmos DB	Azure Cosmos DB	Persistent policy/quote storage	CRUD operations from GQL Service

2. Hook Lifecycle & Execution Flow

Swimlane diagram showing execution order across system layers

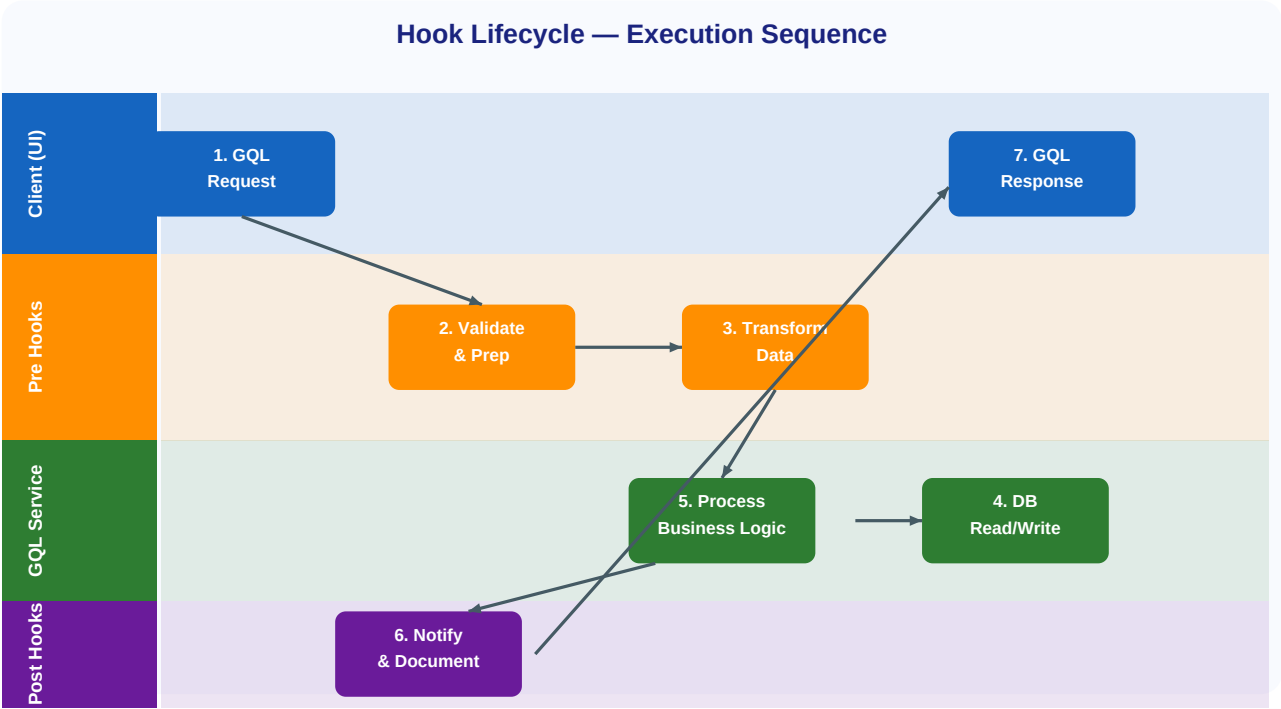


Figure 2: The swimlane diagram shows how control flows vertically between the four layers. Each numbered step represents a discrete phase: the UI initiates the request, Pre Hooks validate and transform it, the GQL Service performs DB operations and business logic, Post Hooks handle notifications and documents, and finally the response travels back to the client.

Execution Phases Detail

Phase	Layer	Actions	Timing
1. GQL Request	Client (UI)	User submits form, GQL mutation/query fired	T+0ms
2. Validate & Prep	Pre Hooks	Schema validation, input sanitization, data enrichment	T+10ms
3. Transform Data	Pre Hooks	Quote number generation, geo-lookup, rule checks	T+20ms
4. DB Read/Write	GQL Service	Cosmos DB CRUD operations via resolver	T+50ms
5. Business Logic	GQL Service	Underwriting rules, pricing, status transitions	T+60ms
6. Notify & Document	Post Hooks	Email, SMS, PDF generation, DMS upload	T+80ms
7. GQL Response	Client (UI)	Policy object returned, UI state updated	T+100ms

3. Hook Schema Structure

Data model and field reference for hook configuration files

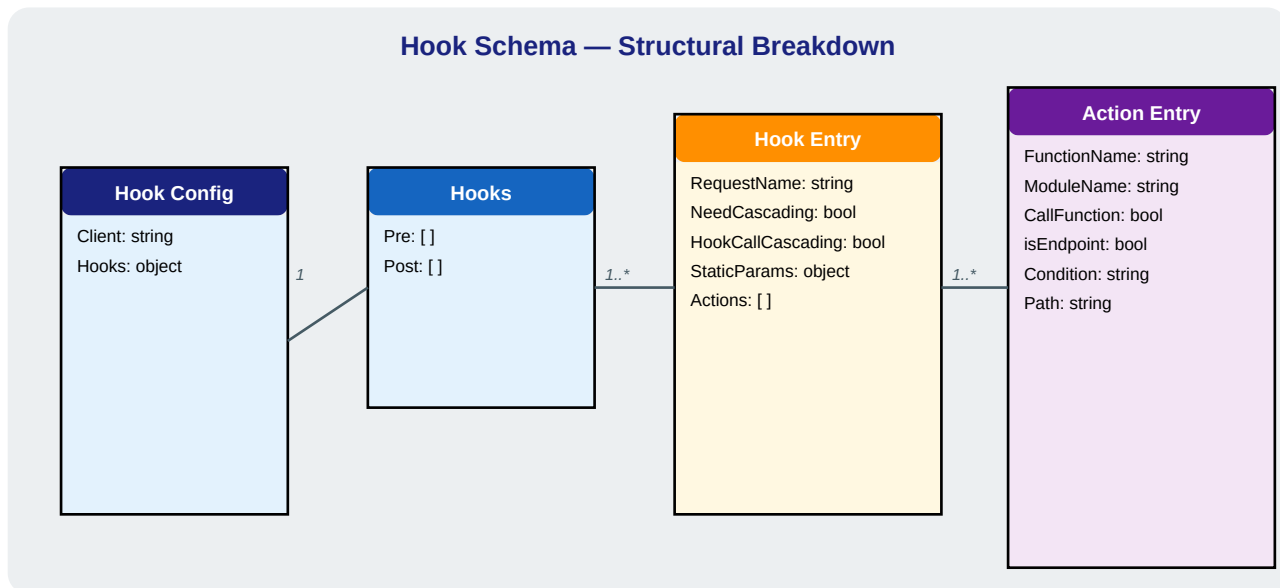


Figure 3: Entity relationship diagram for the Hook Schema JSON. The root object contains a Hooks object with Pre and Post arrays. Each array entry is a Hook Entry with one or more Action Entries defining the function to invoke.

Hook Entry Fields

Field	Type	Required	Description
RequestName	string	Yes	Unique identifier / API endpoint name (e.g. /Quote/Landing)
NeedCascading	boolean	Yes	Whether to cascade operations to related hooks
HookCallCascading	boolean	No	Controls cascading for nested hook executions
StaticParams	object	Yes	Static configuration parameters passed to all actions
Actions	array	Yes	Ordered list of Action Entry objects to execute

Action Entry Fields

Field	Type	Required	Description	Example
FunctionName	string	Yes	Name of function/handler to execute	GenerateQuoteNumber
ModuleName	string	No	npm package containing the function	@cogitate/core-pos-components
CallFunction	boolean	No	Whether to actually invoke the function	true
isEndpoint	boolean	No	Treat action as HTTP/API endpoint call	false
Condition	string	No	Expression that must be true to execute	Transaction.Type = "Application"
Path	string	No	File path for external script execution	BASE/configs/HO3/property.js

4. Visual Workflow Builder UI

Canvas-based drag-and-drop interface design overview

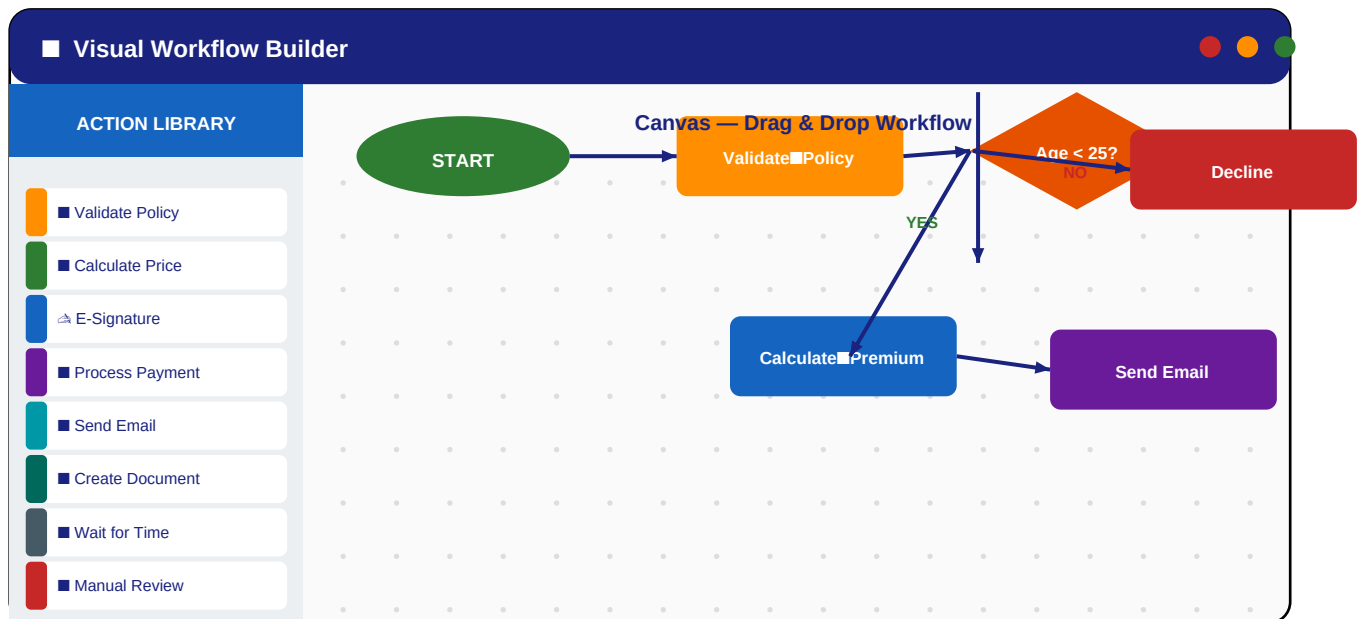


Figure 4: The Visual Workflow Builder presents a sidebar action library on the left and an infinite canvas on the right. Users drag action blocks from the library, connect them with directional arrows, and configure conditions for decision diamonds. The sample workflow shown illustrates a policy validation → age-check decision → premium calculation branch.

Canvas Feature Requirements

ID	Feature	Priority	Description
R-001	Blank Canvas	P0	Large white drawing area visible on open
R-002	Zoom In	P0	Scroll/button to magnify workflow details
R-003	Zoom Out	P0	View complete workflow at reduced scale
R-004	Pan / Drag	P0	Drag canvas to navigate large workflows
R-005	Grid Overlay	P1	Alignment dots or lines for neat block placement
R-006	Auto-expand Canvas	P1	Canvas grows dynamically as blocks are added
R-007	Action Sidebar	P0	Panel listing all draggable action blocks
R-008	Drag to Canvas	P0	Drag action from sidebar onto drawing area
R-014	Color-coded Blocks	P1	Green=action, Blue=data, Orange=decision, etc.
R-020	Branching Arrows	P0	One block can connect to multiple downstream blocks
R-021	Decision Diamond	P0	IF/ELSE node with YES and NO output paths
R-025	Multi-condition	P1	Combine conditions with AND/OR operators

5. Pre & Post Hook Reference

Common hook patterns and when to use each type

Pre Hook Use Cases	Post Hook Use Cases
Generate Quote Number	Send Email Notification
Generate Policy Number	Send SMS Notification
Geo-code Address Lookup	Generate Policy PDF
Execute Underwriting Rules	Upload to Document Store
Call Comparative Rater	Update Policy Status
Validate Input Schema	Trigger Async Web Job
Fetch External Risk Score	Copy / Archive Documents
Prepare Cached Data	Create Audit Log Entry

Condition Expression Examples

Condition Expression	When It Applies
Transaction.Type = 'Application'	Only on new application submissions
Transaction.Type = 'Endorsement' or Transaction.Type = 'Renewal'	Mid-term changes and renewals
Transaction.Type = 'Application' or Transaction.Type = 'Quote'	Both quote and application flows
isMaster = true	Only on primary/master policies (not versions)
ExternalReferences.AIM_QuoteID exists	Skip if AIM ID already assigned

6. GQL Resolver Catalogue

Query and Mutation resolvers — inputs, outputs & event publishing



Key Query Resolvers

Resolver	Input	Output	Description
authenticateUser	client, userName, password	UserDetails	Authenticate via SSO credentials
loadMasters	Client, Table, Role	[MasterData]	Load lookup/master data tables
getByPagination	filters	FilterResponse	Paginated policy list with filters
getComparativeRater	filters	[ComparativePolicy]	Comparative rating data
getById	id	[Policy]	Fetch single policy by ID
getAnalytics	[String] queries	[AnalyticsProperties]	Dashboard analytics data
getTemplate	Carrier, LOB, AgencyId	[Templates]	Fetch form/doc templates

Key Mutation Resolvers (with Event Publishing)

Mutation	Output	Publishes Event	Description
savePolicy	Policy	Yes	Create or update policy record
saveAndBindPolicy	Policy	Yes	Save policy and bind in one operation
bindRenewal	Policy	Yes	Bind renewal to existing policy
copySubmission	Policy	Yes	Duplicate quote/submission
processBillingCancellation	Policy	Yes	Handle billing-triggered cancellation
processBillingReinstatement	Policy	Yes	Reinstate after billing payment
generatePolicyNumber	Policy	No	Assign policy number to bound quote
processOutOfSequence	Policy	No	Handle OOS transaction conflicts
startRenewal	Policy	No	Initialize renewal transaction

7. Available Hook Functions

Function catalogue with parameters and descriptions

invokeAdaptiveAPI

Invokes the Adaptive Form API for document processing. Handles metadata retrieval, template path evaluation, and API header construction.

Parameters:	input, inputModel, extraParams (InputKey, OutputKey, AdaptiveApiURL, ApimSubscriptionKey)
Returns:	object {statusCode, success, data, errors}

getAIMInsuredId

Fetches AIM Insured ID from external API and updates model with external references. Only processes if isMaster=true.

Parameters:	inputModel, extraParams (AimInsuredApiURL, AimApiAuthorization, SubscriptionKey)
Returns:	Updates inputModel.ExternalReferences in-place

getAIMSubmissionId

Fetches AIM Quote ID and adds to model external references. Returns early if AIM_QuoteID already exists.

Parameters:	inputModel, extraParams (AimApiURL, AimApiAuthorization, SubscriptionKey)
Returns:	Updates inputModel.ExternalReferences in-place

ExecuteUnderwritingRules

Runs underwriting rules engine against the policy model. Maps matching rules into model.Rules property.

Parameters:	policy (object), extraParams (RulesEngineURL, SubscriptionKey)
Returns:	Updates policy.Rules array

generateFormsDraft

Prepares forms requirements based on product, LOB, and transaction type for PDF generation.

Parameters:	policy, extraParams (productCode, transactionType, programType, isDraft, policyEffectiveDate)
Returns:	string (draft ID or status)

invokeMultipleRaters

Concurrently invokes multiple rating engines and consolidates results.

Parameters:	listOfConfigs (array: RaterName, RaterURL, RaterKey, RaterType), extraParams.input
Returns:	Consolidated rating results object

processOOS

Processes Out-of-Sequence transactions. Handles binding, version management, and status updates.

Parameters:	policy, extraParams (userInfo, Client)
Returns:	Updated policy object

createQuoteVersion

Creates a new version of an existing quote with version tracking.

Parameters:	policy (QuoteNumber, QuoteVersion), extraParams (ClientCode, MaxAllowedQuotes, userInfo)
Returns:	New policy version object

deleteQuoteVersionsOnBind

Cleans up intermediate quote versions when a quote is bound.

Parameters:	policy (version info), extraParams (userInfo)
Returns:	Void — modifies DB records directly

8. Workflow Requirements Matrix

Full feature requirements from the Visual Workflow Builder spec

ID	Feature Area	Requirement	Priority
R-001	Canvas	Show blank white canvas on open	P0
R-002	Canvas	Zoom in to see block details	P0
R-003	Canvas	Zoom out to see full workflow	P0
R-004	Canvas	Pan by dragging canvas	P0
R-005	Canvas	Display alignment grid	P1
R-006	Canvas	Auto-expand canvas as blocks added	P1
R-007	Library	Sidebar showing all available actions	P0
R-008	Library	Drag action block onto canvas	P0
R-009	Library	Drop block at any canvas position	P0
R-010	Library	Show icon + name on each block	P1
R-011	Blocks	Move blocks after placing them	P0
R-012	Blocks	Delete blocks	P0
R-013	Blocks	Copy/duplicate a block	P1
R-014	Blocks	Color-code by block type	P1
R-015	Connections	Draw directional arrows between blocks	P0
R-016	Connections	Click-drag to connect blocks	P0
R-017	Connections	Show arrow direction clearly	P0
R-018	Connections	Delete connection arrows	P0
R-019	Connections	Auto-curve arrows around blocks	P1
R-020	Connections	One block → multiple blocks (branching)	P0
R-021	Decisions	Add IF/ELSE decision diamond block	P0
R-022	Decisions	Write condition in plain language	P0
R-023	Decisions	Show YES and NO output paths	P0
R-024	Decisions	Connect different blocks to each path	P0
R-025	Decisions	Multi-condition (AND/OR) support	P1
R-029	Library	Categorized action browser	P1
R-030	Library	Search actions by name	P0
R-031	Library	Filter actions by category	P1
R-035	Pre-built	Validate Policy Data action	P0
R-036	Pre-built	Calculate Insurance Price action	P0
R-037	Pre-built	Run Underwriting Rules action	P0
R-038	Pre-built	Send for E-Signature action	P0

R-039	Pre-built	Process Payment action	P0
R-040	Pre-built	Check Risk Score (external) action	P1
R-041	Pre-built	Update Policy Status action	P0
R-042	Pre-built	Send Email Notification action	P0
R-043	Pre-built	Send SMS Notification action	P1
R-044	Pre-built	Create Policy Document (PDF) action	P0
R-047	Pre-built	Wait for Time (timer) action	P1
R-048	Pre-built	Send to Manual Review action	P1

9. Hook Schema Quick Reference

Annotated sample JSON for building new hook configurations

Minimal Hook Schema Template

```
{
  "Client": "YOUR_CLIENT_CODE",
  "Hooks": {
    "Pre": [
      {
        "RequestName": "/Quote/Landing",           // Hook trigger endpoint
        "NeedCascading": true,                     // Cascade to related hooks
        "HookCallCascading": true,                 // Cascade nested hook calls
        "StaticParams": {},                        // Config params for all actions
        "Actions": [
          {
            "FunctionName": "GenerateQuoteNumber", // Function to call
            "ModuleName": "@cogitate/core-pos-components", // npm module
            "CallFunction": true,                  // Actually call it
            "isEndpoint": false,                   // Not an HTTP call
            "Condition": "Transaction.Type = 'Quote'", // Guard condition
            "Path": ""                             // No external script
          },
          {
            "FunctionName": "getGeoCodeAddressHook",
            "ModuleName": "@cogitate/core-pos-components",
            "CallFunction": true,
            "isEndpoint": false,
            "Condition": "",
            "Path": "BASE/configs/H03/property.js"
          }
        ]
      }
    ],
    "Post": [
      {
        "RequestName": "/Application/Summary",
        "NeedCascading": false,
        "StaticParams": {},
        "Actions": [
          {
            "FunctionName": "EmailHook",
            "ModuleName": "@cogitate/core-pos-components",
            "CallFunction": true,
            "isEndpoint": false,
            "Condition": "Transaction.Type = 'Application'",
            "Path": ""
          }
        ]
      }
    ]
  }
}
```

Condition Expression Reference

Pattern	Syntax	Example
Equality	field = 'value'	Transaction.Type = 'Application'
OR condition	expr OR expr	Transaction.Type = 'Quote' or Transaction.Type = 'Application'

AND condition	expr AND expr	Transaction.Type = 'Application' and isMaster = true
Exists check	field exists	ExternalReferences.AIM_QuoteID exists
Boolean flag	field = true/false	isMaster = true

10. Implementation Checklist

Step-by-step guide for developers starting the build

Phase 1: Foundation

- Set up Next.js project with Apollo Client for GQL
- Configure GQL Service with Apollo Server
- Establish Cosmos DB connection and schema
- Deploy basic Hook middleware pipeline
- Validate end-to-end request flow with a simple test resolver

Phase 2: Hook Engine

- Implement Pre Hook loader — reads hook schema JSON
- Implement Post Hook loader — reads hook schema JSON
- Build Condition evaluator (parse and evaluate condition strings)
- Implement function dispatcher (ModuleName + FunctionName → call)
- Add isEndpoint support for HTTP action calls
- Add Path-based external script execution
- Write unit tests for each hook function in the catalogue

Phase 3: Workflow Builder UI

- Set up React Flow (or similar) canvas library
- Build Action Library sidebar with categories and search
- Implement drag-and-drop from sidebar to canvas (R-008/R-009)
- Build connection logic — click-drag arrows between blocks (R-015/R-016)
- Implement Decision Diamond block with YES/NO branching (R-021)
- Add block deletion and copy functionality (R-012/R-013)
- Implement Zoom In / Zoom Out / Pan controls (R-002/R-003/R-004)
- Add grid overlay for alignment (R-005)
- Build canvas → Hook Schema JSON serializer
- Build Hook Schema JSON → canvas deserializer (for loading saved workflows)

Phase 4: Pre-built Actions

- Wire up all pre-built action blocks (R-035 to R-048)

- Connect each block to its corresponding hook function
- Add block configuration panels (input params UI)
- Integrate external APIs (risk score, geo-code, AIM, raters)
- End-to-end test: Quote → Application → Bind policy flow

Phase 5: Polish & Deploy

- Add workflow validation (detect disconnected blocks, missing conditions)
- Implement save/load workflow to Cosmos DB
- Add undo/redo history for canvas actions
- Performance testing with large workflows (50+ blocks)
- Security review: condition injection, API key management
- User acceptance testing with insurance domain experts